

2048 CLI

20240131 Seoho Kim

May 3, 2026

1 Overview

This project is a command-line implementation of the sliding-tile puzzle game **2048**, written entirely in **F# with .NET 10**. The player slides tiles on a square board using the W/A/S/D keys. When two tiles of the same value collide, they merge into a single tile whose value is their sum. The player wins when a tile reaches the value 2048. The game is lost when no legal move remains.

The player may choose from four board sizes before each game: 2×2 , 4×4 , 8×8 , and 16×16 . The per-size all-time high score is persisted to a local file between sessions.

The implementation follows a FUNCTIONAL CORE and IMPERATIVE SHELL architecture: all game logic (tile merging, move validation, win/loss detection) is expressed as pure, immutable functions with no mutable state, while console I/O is confined to the outermost layer.

2 Requirements

2.1 Main Menu

R1. When the program starts, it displays a main menu containing exactly four options:

1) 3×3 , 2) 4×4 , 3) 5×5 , 4) 6×6 , 5) 8×8 .

R2. The user selects a board size by entering the digit 1, 2, 3, 4, or 5. If any other input is entered, the menu is re-displayed and the user is prompted again.

R3. The main menu also displays the all-time high score for each board size, loaded from the local score file. If no score has been recorded for a size, it is displayed as 0.

2.2 Game Initialisation

R4. After the user selects a board size, a new game board of that size is created with all cells initialised to 0.

- R5.** Exactly two tiles are placed on the board in randomly chosen distinct empty cells before the first move. Each tile's value is independently sampled: 2 with probability 0.9 and 4 with probability 0.1.
- R6.** The current score is initialised to 0.

2.3 Display

- R7.** After every state change (initialisation, successful move, win, or loss), the board is re-rendered to the terminal.
- R8.** Each cell is displayed in a consistent format such that all cells have equal width. The exact formatting, padding, and representation of empty cells are implementation-defined.
- R9.** The current score and the all-time high score for the selected board size are displayed on every render.
- R10.** The available controls (W/A/S/D to move, R to restart, Q to quit) are displayed on every render.

2.4 Moves

- R11.** During a game the program reads exactly one character of input at a time without requiring the user to press Enter (raw keypress mode). Valid move keys are W (up), A (left), S (down), D (right), case-insensitively.
- R12.** A *slide* in a given direction is computed row-by-row (for left/right) or column-by-column (for up/down). Within each line, the following steps are applied in order:
- (a) Remove all zero cells, compacting non-zero values toward the leading edge.
 - (b) Scan the compacted sequence from the leading edge: whenever two adjacent values are equal, replace them with a single cell whose value is their sum. Each cell participates in at most one merge per move.
 - (c) Pad the resulting sequence with zeros at the trailing edge to restore the original line length.
- R13.** When two tiles merge, their sum is added to the current score.
- R14.** If the resulting board after a slide is identical to the board before the slide, the move is considered *ineffective*. An ineffective move does not place a new tile and does not change the score.
- R15.** If a move is effective, one new tile is placed in a randomly chosen empty cell. The tile value is 2 with probability 0.9 and 4 with probability 0.1.

2.5 Win Condition

- R16.** If any cell on the board contains the value 2048 after a move, the game enters the *win* state.
- R17.** Upon entering the win state, the board and score are rendered, followed by the message **You win!**
- R18.** The current score is compared with the stored high score for the selected board size. If the current score is greater, the high score is updated in the local score file.
- R19.** After displaying the win message, the program waits for any keypress, then returns to the main menu.

2.6 Loss Condition

- R20.** After every effective move and tile placement, the program checks whether any legal move remains. A legal move exists if and only if at least one of the four slides would produce a board different from the current board.
- R21.** If no legal move exists, the game enters the *loss* state.
- R22.** Upon entering the loss state, the board and score are rendered, followed by the message **Game over.**
- R23.** The high score update rule in R17 applies identically on loss.
- R24.** After displaying the game-over message, the program waits for any keypress, then returns to the main menu.

2.7 In-Game Controls

- R25.** Pressing R (case-insensitively) during a game immediately ends the current game without updating the high score, and returns the player to the main menu.
- R26.** Pressing Q (case-insensitively) during a game immediately ends the current game without updating the high score, and exits the program.
- R27.** Any key other than W, A, S, D, R, Q is silently ignored; the board is not re-rendered.

2.8 Score Persistence

- R28.** High scores are stored in a plain-text file named `scores.txt` in the same directory as the executable.
- R29.** The file contains one line per board size in the format `<size>:<score>`, e.g. `4:13204`. Board sizes not yet recorded are omitted; their high score is treated as 0.

R30. If `scores.txt` does not exist at program startup, it is treated as empty (all high scores are 0). The file is created when the first high score is written.

3 Example Interaction

Note

The following interaction is illustrative. The exact visual layout may differ in the final implementation.

The program starts and displays:

```
=== 2048-CLI ===
Select board size:
  1) 2x2    (Best: 0)
  2) 4x4    (Best: 0)
  3) 8x8    (Best: 0)
  4) 16x16 (Best: 0)
> _
```

The user presses 2. A 4×4 board is created, two tiles are placed, and the following is rendered:

```
Score: 0    Best: 0
. . . 2
. . . .
. 4 . .
. . . .
[W/A/S/D] Move  [R] Restart  [Q] Quit
```

The user presses A (slide left). Tiles move left; a new tile appears in a random empty cell. The updated board and score are displayed.

4 Out of Scope

The following features are explicitly *not* part of this project's requirements and will not be implemented:

- Graphical or browser-based rendering.
- Online or networked leaderboards.
- Undo functionality.

- Animations or sound effects.
- Targets other than 2048 (e.g., a “continue past 2048” mode).

5 Repository

<https://github.com/seohokim-hoya/2048-cli>